



Chapter 14

Big Data and NoSQL



Learning Objectives

- **After completing this chapter, you will be able to:**
 - Explain the role of Big Data in modern business
 - Describe the primary characteristics of Big Data and how these go beyond the traditional “3 Vs”
 - Explain how the core components of the Hadoop framework operate
 - Identify the major components of the Hadoop ecosystem
 - Summarize the four major approaches of the NoSQL data model and how they differ from the relational model
 - Describe the characteristics of NewSQL databases
 - Understand how to work with document databases using MongoDB
 - Understand how to work with graph databases using Neo4j



Big Data

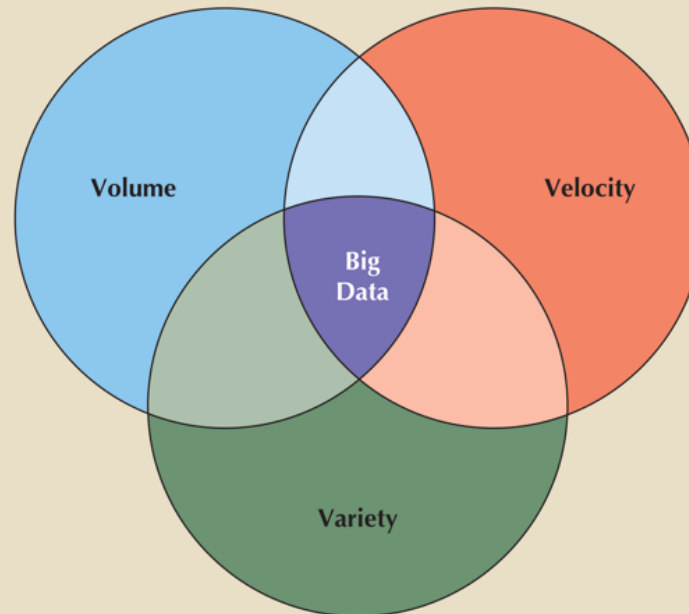
- One way to think about **Big data** is as a new way of data storage and manipulation possibilities and requirements
- Conventionally, **Big Data** is a set of data that displays the characteristics of volume, velocity and variety (3Vs)
 - These characteristics make the data unsuitable for management by a relational database management system
- Recently, big data includes **veracity**, resulting into 4Vs
- Some even add **value**, leading to 5Vs
- **Official definition:**
 - Big data is a collection of multi-structured, semi-structured and unstructured data from traditional and digital sources inside and outside an organization that represents a source for ongoing discovery and analysis



Original view of Big Data

- **Volume**: the quantity of data to be stored
- **Velocity**: the speed at which data is entering the system
- **Variety**: different format and structure from diverse sources

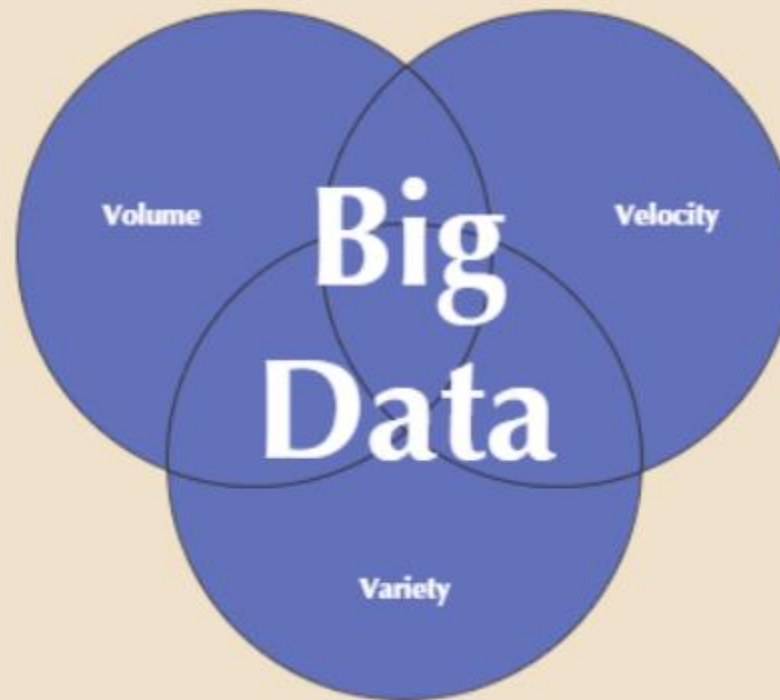
FIGURE 14.1 ORIGINAL VIEW OF BIG DATA





Current view of Big Data

FIGURE 14.2 CURRENT VIEW OF BIG DATA





Volume

- Refers to the quantity to be stored
- As the quantity of data needing to be stored increases, the need for larger storage devices increases as well
 - Systems can either scale up or scale out
 - **Scaling up:** keeping the same number of systems, but migrating each system to a larger storage system
 - **Scaling out:** means that when the workload exceeds the capacity of a server, the workload is spread out across a number of servers
 - Sometimes called **clustering** – creating a cluster of low-cost servers to share a workload
 - RDBMS is not suited for clusters because of increased performance costs of communication and coordination



Volume

TABLE 14.1

STORAGE CAPACITY UNITS

TERM	CAPACITY	ABBREVIATION
Bit	0 or 1 value	b
Byte	8 bits	B
Kilobyte	1024* bytes	KB
Megabyte	1024 KB	MB
Gigabyte	1024 MB	GB
Terabyte	1024 GB	TB
Petabyte	1024 TB	PB
Exabyte	1024 PB	EB
Zettabyte	1024 EB	ZB
Yottabyte	1024 ZB	YB

* Note that because bits are binary in nature and are the basis on which all other storage values are based, all values for data storage units are defined in terms of powers of 2. For example, the prefix *kilo* typically means 1000; however, in data storage, a kilobyte = 2^{10} = 1024 bytes.



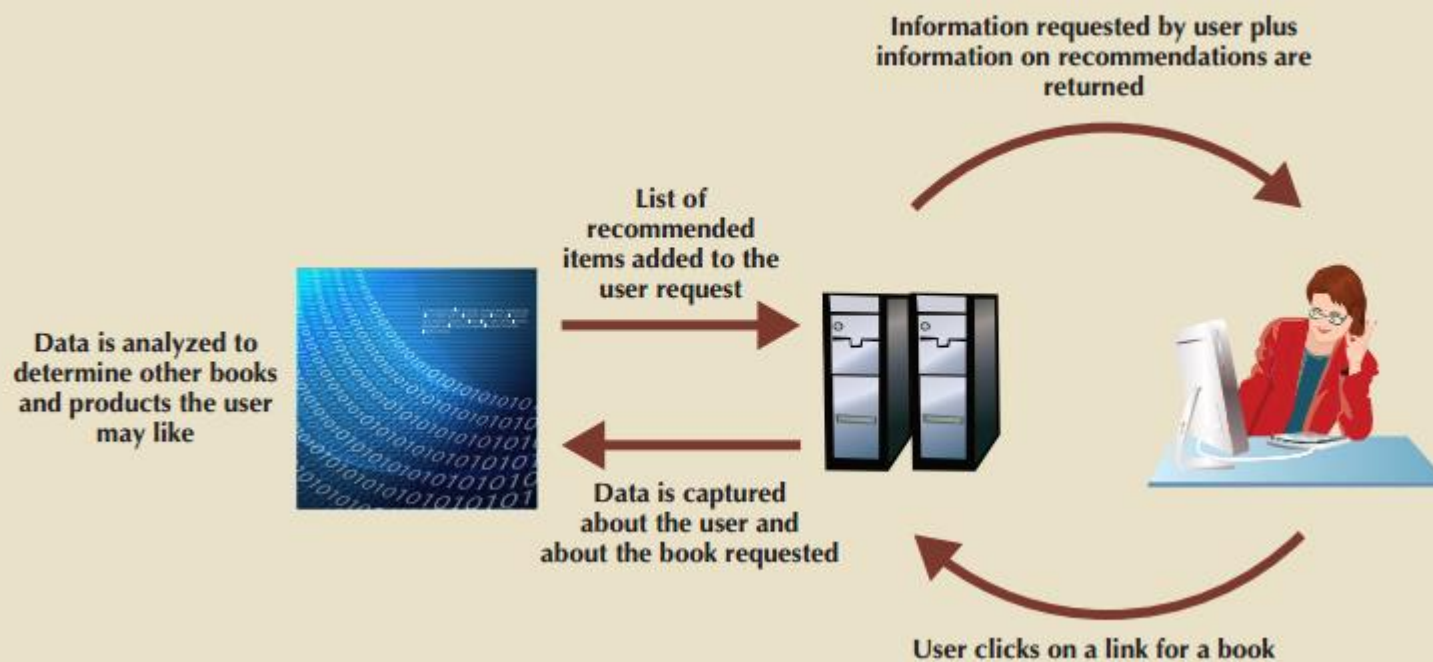
Velocity

- Refers to the rate at which new data enters and get processed in a system
- The velocity of processing can be broken down into two categories
 - **Stream processing**: focuses on input processing and requires analysis of data stream as it enters the system
 - **Feedback loop processing**: analysis of data to produce actionable results.
 - Can be thought of as focused on outputs



Big Data

FIGURE 14.3 FEEDBACK LOOP PROCESSING





Variety

- Refers to the vast array of formats and structures in which the data may be captured
 - **Structured data**: fits into a predefined data model
 - **Unstructured data**: does not fit into a predefined model
 - Examples include maps, satellite images, emails, texts, tweets, videos, transcripts, and a whole host of other data forms
 - **Semi-structured data**: combines elements of both structured and unstructured data.



Binary large object (BLOB) data type supported by RDBMS

- Relational databases have evolved to address some forms of unstructured data
 - Most large-scale RDBMSs support a binary large object (BLOB) data type
 - BLOB allows the storage of unstructured objects like audio, video, and graphic data as a single, atomic value
 - Problem with BLOB data storage:
 - The semantic value of the data, i.e., the meaning that the object conveys, is inaccessible and uninterpretable by data processing



Big Data

- **Other characteristics of Big Data**

- **Variability**: changes in meaning of data based on context
 - **Relevant in areas such as Sentimental analysis that** attempts to determine if a statement conveys a positive, negative or neutral attitude about a topic
- **Veracity**: trustworthiness of data
- **Value (viability)**: degree to which the data can be analysed to provide meaningful information that can add value to the organization
- **Visualization**: ability to graphically present data to make it understandable
- Relational databases are not necessarily the best for storing and managing all organizational data
 - **Polyglot persistence**: coexistence of a variety of data storage and management technologies within an organization's infrastructure



BIG DATA ENABLING TECHNOLOGIES



1. Hadoop

- De facto standard for most Big Data **storage** and **processing**
 - Java-based framework for distributing and processing very large data sets across clusters of computers
- **Its important components:**
 - **Hadoop Distributed File System (HDFS):** low level, highly distributed, fault-tolerant file storage and processing system designed to manage large amounts of data at high speeds
 - **MapReduce:** programming model that supports processing large data sets in a highly parallel, distributed manner



Hadoop

- HDFS approach to distributing data is based on several key assumptions:
 - **High volume**: default block sizes is 64 MB and can be configured to even larger values
 - **Write-once, read-many**: model simplifies concurrency issues and improves data throughput
 - **Streaming access**: optimized for batch processing of entire files as a continuous stream of data
 - **Fault tolerance**: designed to replicate data across many different devices so that when one fails, data is still available from another device



Hadoop

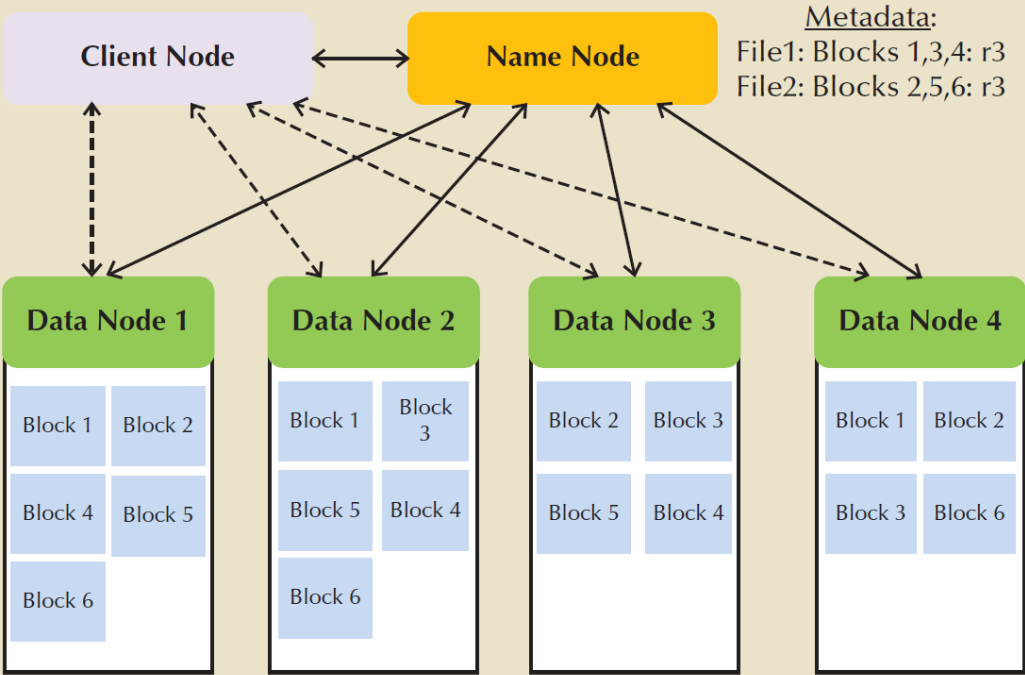
- **Hadoop uses several types of nodes**; computers that perform one or more types of tasks within the system
 - **Data node** store the actual file data and communicates with name node and send back block reports and heartbeats
 - **Name node** contains file system metadata
 - **Client node** makes requests to the file system as needed to support user applications



HDFS's Nodes

Block report sent to the name node every 6 hours and informs the name node of which blocks are on that data node

FIGURE 14.4 HADOOP DISTRIBUTED FILE SYSTEM (HDFS)



Heartbeat sent to the name node every 3 seconds, used to let the name node know that the data node is still available



Name Node

- Contains metadata for file systems
 - Designed to be small, simple, easily recoverable
- Only one name node exist within a HDFS
- The name node comprise of the following:
 - Each file Name
 - Block numbers
 - Desired replication factor



Data Node

- Is responsible for actual file data within the HDFS
- The files are broken into blocks
- Files are also replicated to ensure fault tolerance
- Each block is duplicated on more than one data node



HDFS process

Client File Make request to name node

The name node do the following

- **Adds the file name to metadata**
- **Determines a new block number for file**
- **Determines a list of data nodes, where the block/files will be stored.**
- **Passes the information back to the client node**

Client Node contacts the first data node specified by name node and begins writing on that node



Hadoop

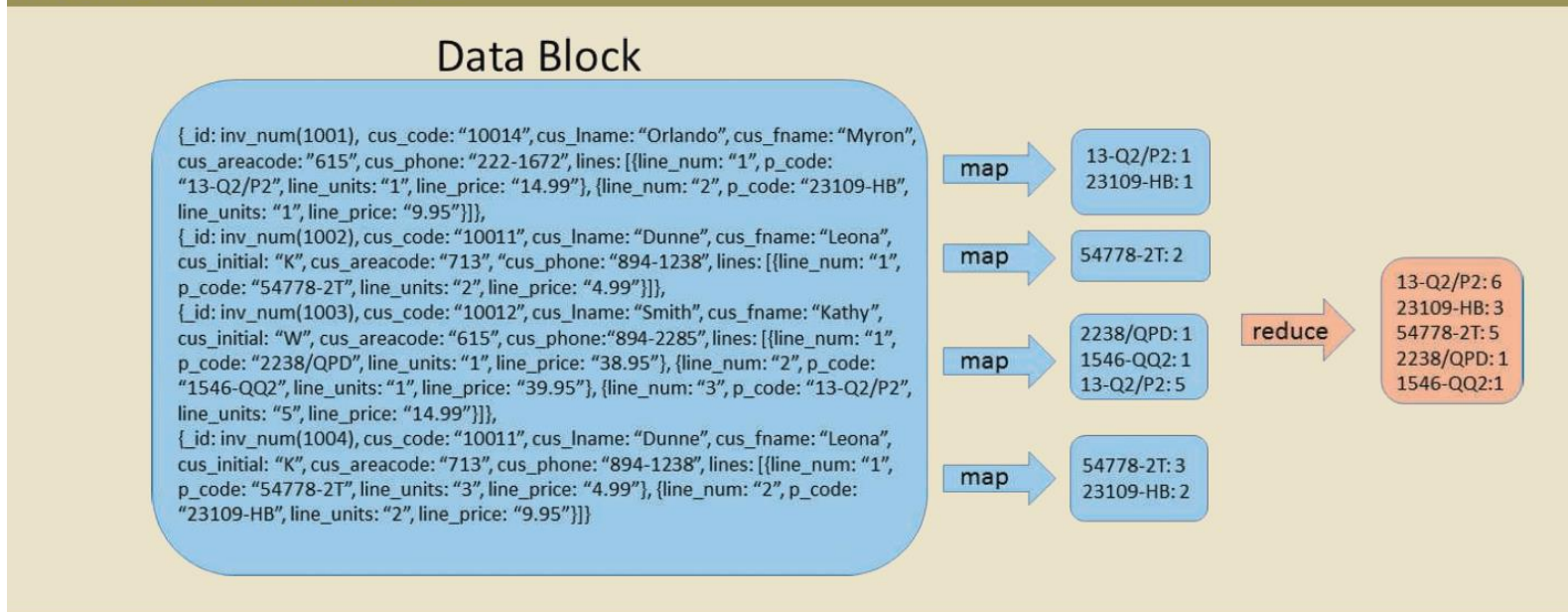
- MapReduce

- Framework used to process large data sets across clusters
 - Breaks down complex tasks into smaller subtasks, performing the subtasks and producing a final result
 - **Map function** takes a collection of data, sorts and filters it into a set of key-value pairs
 - The map function is performed by a program called a **mapper**
 - **Reduce function** takes a collection of key-value pairs, all with the same key value and summarizes them into a single result
 - The reduce function is performed by a program called a **reducer**



MapReducer Example

FIGURE 14.5 MAPREDUCE



Job Tracker: acts as a central control for MapReduce processing, by locating the data

Task Tracker (reside in a data node): is responsible for creating a new JVM to run the map and reduce functions



Batch processing

- The Hadoop system uses **batch processing**
 - Means a program runs from beginning to end, either completing the task or halting with an error, without any interaction with the user
- Often used when the **computing task** requires an extended period of time or a large portion of the system's processing capacity
- **Batch processing has limitations**
 - As a result, a number of complementary programs have been developed to improve the integration of Hadoop within the larger IT infrastructure



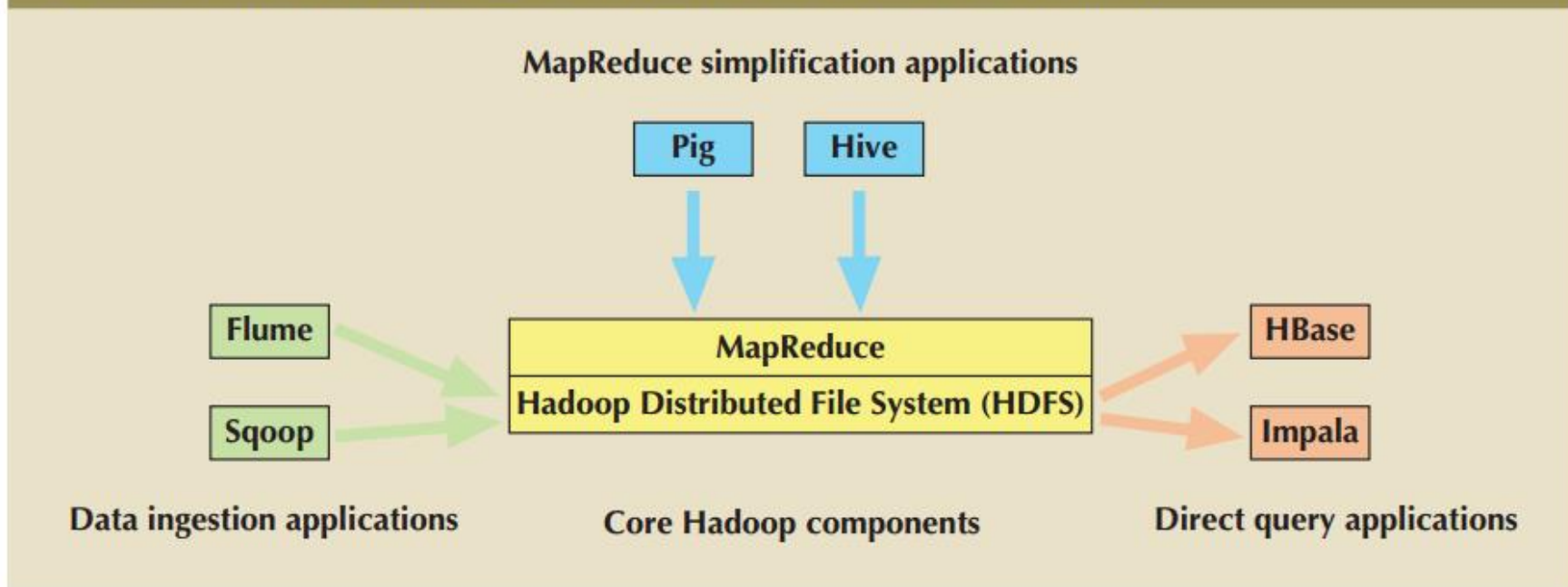
Hadoop ecosystem

- Hadoop is a very low-level tool
 - Requires considerable effort to create, manage and use it
 - A host of related applications have grown up around Hadoop. These applications attempt to make Hadoop
 - easier to use and
 - more accessible to users who are not skilled at complex Java programming
- The next slide shows examples of some of these types of applications in a Hadoop ecosystem



Hadoop ecosystem

FIGURE 14.6 A SAMPLE OF THE HADOOP ECOSYSTEM





Hadoop ecosystem

- **Map reduce simplification applications**
 - **Hive**: data warehousing system that sits on top of HDFS and supports its own SQL-like language called HiveQL
 - **Pig**: tool that compiles a high-level scripting language, named Pig Latin, into MapReduce jobs for executing in Hadoop
- **Data ingestion applications**
 - **Flume**: component for ingesting data in Hadoop. Designed primarily for harvesting large sets of data from server log files
 - **Sqoop**: tool for converting data back and forth between a relational database and the HDFS
- **Direct query applications**
 - **Hbase**: column-oriented NoSQL database designed to sit on top of the HDFS that quickly processes sparse datasets
 - **Impala**: the first SQL on Hadoop application



2. NoSQL

- **Nosql**: non-relational database technologies developed to address Big Data challenges
- Name does not describe what the NoSQL technologies are, but rather what they are not

NoSQL DATABASES	
NoSQL CATEGORY	EXAMPLE DATABASES
Key-value database	Dynamo Riak Redis Voldemort
Document databases	MongoDB CouchDB OrientDB RavenDB
Column-oriented databases	HBase Cassandra Hypertable
Graph databases	Neo4J ArangoDB GraphBase



NoSQL: Key-Value (KV) Database

- The simplest of the NoSQL data models
- Stores data as collectives of keys
- Has no foreign keys (**no relationship among keys**)
 - Making KV databases fast and scalable
- Pairs are organised as buckets, an equivalent of tables
- Operations are limited to **get, store, and delete**
 - **Get or fetch** is used to retrieve the value component of the pair.
 - **Store** is used to place a value in a key

FIGURE 14.7 KEY-VALUE DATABASE STORAGE

Bucket = Customer

Key	Value
10010	"LName Ramas FName Alfred Initial A Areacode 615 Phone 844-2573 Balance 0"
10011	"LName Dunne FName Leona Initial K Areacode 713 Phone 894-1238 Balance 0"
10014	"LName Orlando FName Myron Areacode 615 Phone 222-1672 Balance 0"



NoSQL: Document database

- Similar to KV database, sometimes considered a subtype of KV databases
- Stores data in tagged documents in **key-value pairs**
- Value components are encoded documents grouped into large groups called **collections**
- The document database attempt to understand the content

FIGURE 14.8 DOCUMENT DATABASE TAGGED FORMAT

Collection = Customer

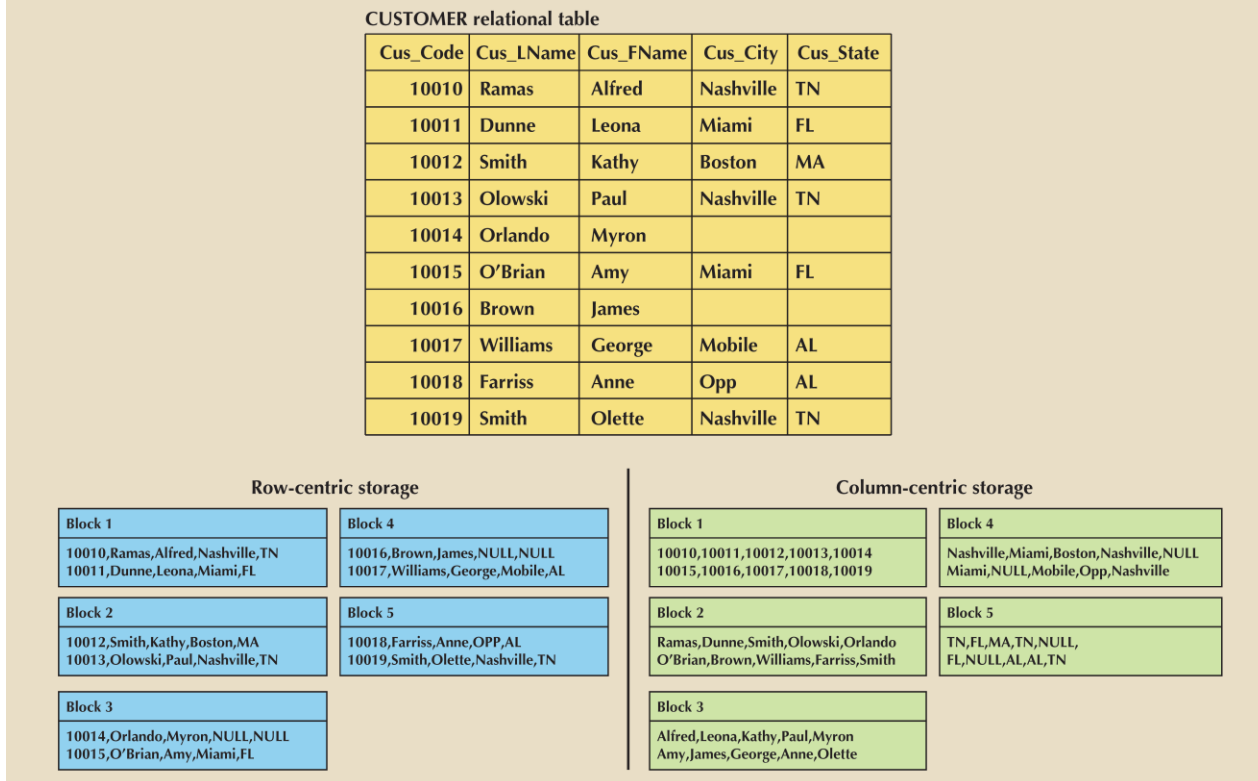
Key	Document
10010	{LName: "Ramas", FName: "Alfred", Initial: "A", Areacode: "615", Phone: "844-2573", Balance: "0"}
10011	{LName: "Dunne", FName: "Leona", Initial: "K", Areacode: "713", Phone: "894-1238", Balance: "0"}
10014	{LName: "Orlando", FName: "Myron", Areacode: "615", Phone: "222-1672", Balance: "0"}



NoSQL: Column oriented database

- Refers to 2 technologies:
- Column-centric storage: data stored in blocks which hold data from a single column across many rows
- A single customers data is spread across several blocks
- Row-centric storage: data stored in block which hold data from all columns of a given set of rows
- Can be achieved within relational databases

FIGURE 14.9 COMPARISON OF ROW-CENTRIC AND COLUMN-CENTRIC STORAGE

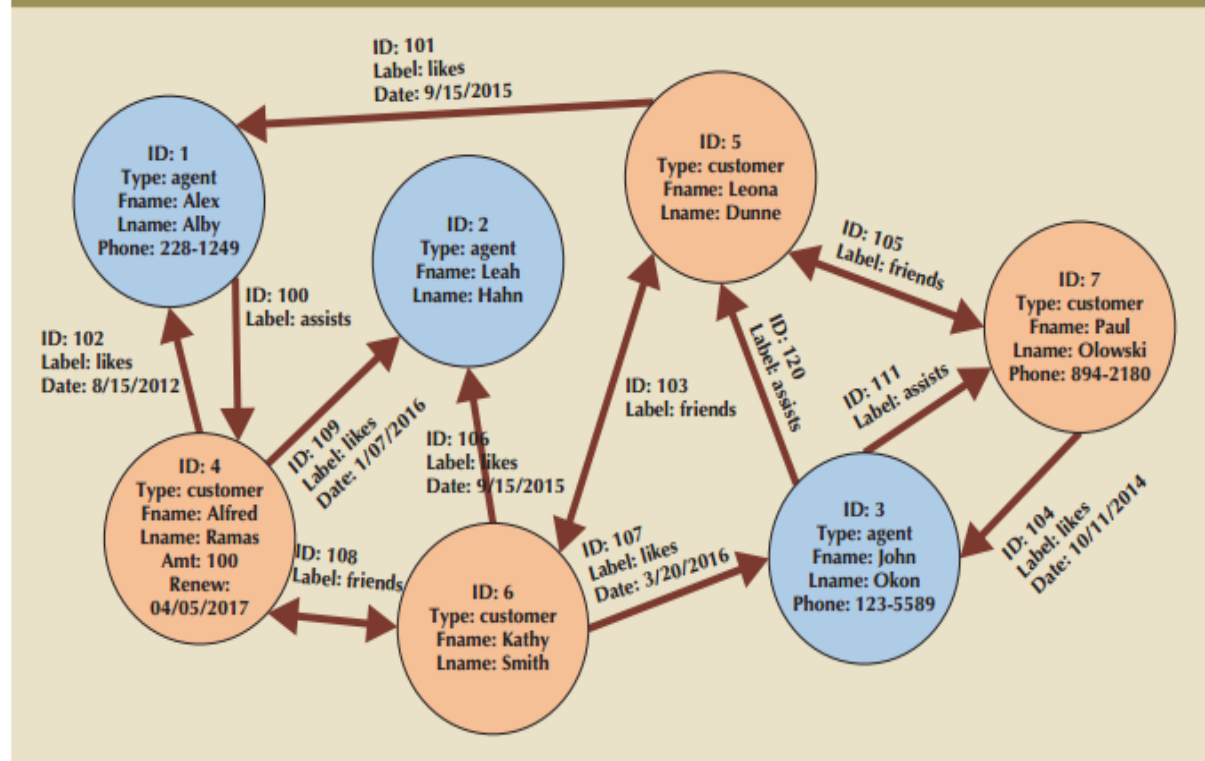




NoSQL: Graph database

- Graph database is based on **graph theory** to store data about **relationship-rich environment**
- Graph theory is mathematical and computer science field that models **relationships between nodes**
- The primary components of graph database are **nodes, edges and properties**
- A **node** is a specific instance of something to store data about
- An **edge** (unidirectional/bidirectional) is a relationship between nodes
- **Properties** are attributes of either the node or edge
- A query is called a **traversal**

FIGURE 14.11 GRAPH DATABASE REPRESENTATION





Aggregate awareness

- **Aggregate awareness**: data is collected or aggregated around a central topic or entity
 - **Aggregate aware database models** achieve clustering efficiency by making each piece of data relatively independent
 - That allows a key-value pair to be stored on one node in the cluster
- **Graph databases, like relational databases, are aggregate ignorant**
 - Do not organize the data into collections based on a central entity
- **Key-value, document and column family databases are aggregate aware**



3. NewSQL Databases

- Database model that attempts to provide ACID-compliant transactions across a highly distributed infrastructure
 - Latest technologies to appear in the data management area to address Big Data problems
 - No proven track record
 - Have been adopted by relatively few organizations
 - Attempts to bridge the gap between SQL and NoSQL databases



NewSQL Databases

- Like RDBMS, NewSQL databases support:
 - SQL as the primary interface
 - ACID-compliant transactions
- Similar to NoSQL, NewSQL databases also support:
 - Highly distributed clusters
 - Key-value or column-oriented data stores



Working with Document Databases Using MongoDB

- Popular document database
 - Among the NoSQL databases currently available, MongoDB has been one of the most successful in penetrating the database market
- MongoDB, comes from the word humongous as its developers intended their new product to support extremely large data sets. It is designed for:
 - High availability
 - High scalability
 - High performance



Working with Document Databases Using MongoDB

- Importing Documents in MongoDB
 - Refer to the text for an importation example and considerations
- **Example of a MongoDB Query Using find()**
 - Methods are programmed functions to manipulate objects
 - **Find()** method retrieves objects from a collection that match the restrictions provided
 - **Pretty()** method is used to improve readability of the documents by placing key:value pairs on separate lines
 - Refer to the text for a query example



Working with Graph Databases Using Neo4j

- Even though Neo4j is not yet as widely adopted as MongoDB, it has been one of the fastest growing NoSQL databases
 - Graph databases still work with concepts similar to entities and relationships
 - Focus is on the relationships
 - Graph databases are used in environments with complex relationships among entities
 - Heavily reliant on interdependence among their data
 - Neo4j provides several interface options
 - Designed with Java programming in mind
- Creating nodes in Neo4j
 - Nodes in a graph database correspond to entity instances in a relational database
 - Cypher is the interactive, declarative query language in Neo4j
 - Nodes and relationships are created using a CREATE command



Working with Graph Databases Using Neo4j

- Using the **CREATE** command to create a member node, e.g.,

```
CREATE (:Member {mid: 1, fname:
  "Sibusiso",      lname: "Dlamini"})
```

- The command creates a node with the Member label.
- Retrieving node data with **MATCH** and **WHERE**

```
MATCH (m)
WHERE m.fname = "Sibusiso" AND m.lname = "Dlamini"
RETURN m
```



Working with Graph Databases Using Neo4j

- It is also possible to retrieve data based on the relationships between nodes
 - Retrieving relationship data with **MATCH** and **WHERE**
 - **E.g.**, the following command retrieves every member who has reviewed the restaurant “Nandos Mbombela” and rated the restaurant a “4” on taste

```
MATCH (m :Member) -[r :REVIEWED {taste: 4}]-> (res
      :Restaurant {name: "Nandos Mbombela"})
RETURN m, r, res
```

- In this example, there are two nodes (**m** and **res**) and a relationship that joins them (**r**)
- In this case, we are matching all nodes that are members, the one node that is named “Nandos Mbombela” and all relationships that are labelled as REVIEWED and have a property named “taste” equal to the value “4”



Summary

- Big Data is characterized by data of such volume, velocity, and/or variety that the relational model struggles to adapt to it
- Volume, velocity, and variety are collectively referred to as the 3 Vs of Big Data
- The Hadoop framework has quickly emerged as a standard for the physical storage of Big Data
- NoSQL is a broad term to refer to any of several nonrelational database approaches to data management
- Key-value databases store data in key-value pairs
- Document databases also store data in key-value pairs, but the data in the value component is an encoded document



Summary

- Column-oriented databases, also called column family databases, organize data into key-value pairs in which the value component is composed of a series of columns, which are themselves key-value pairs
- Graph databases are based on graph theory and represent data through nodes, edges, and properties
- NewSQL databases attempt to integrate features of both RDBMS (providing ACID-compliant transactions) and NoSQL databases (using a highly distributed infrastructure)
- MongoDB is a document database that stores documents in JSON format
- Neo4j is a graph database that stores data as nodes and relationships, both of which can contain properties to describe them